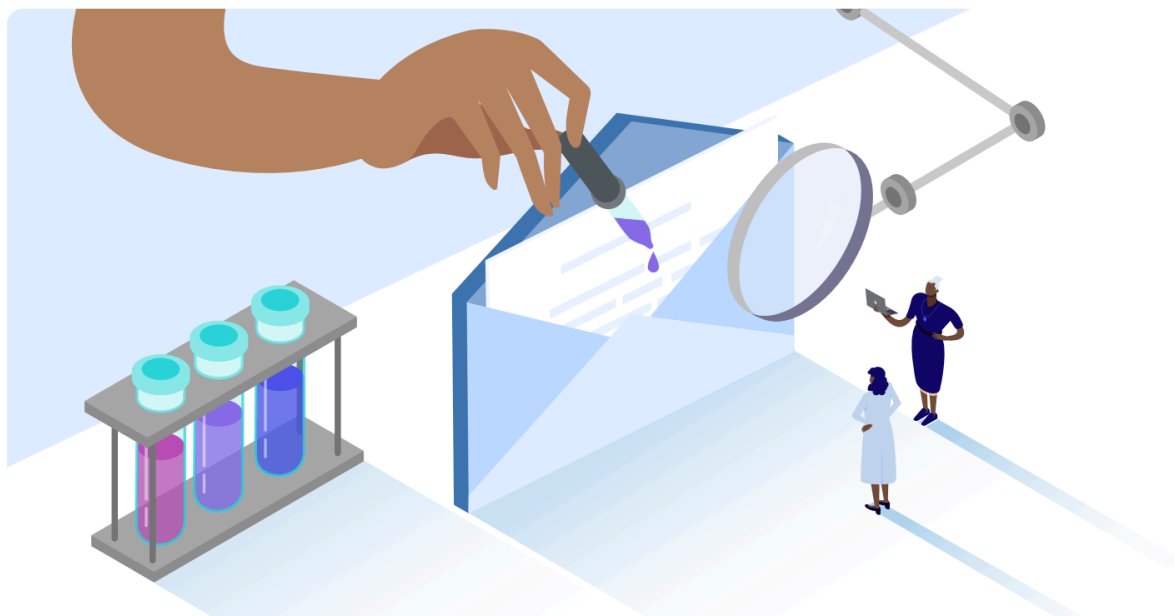


Советы по email-маркетингу

Как использовать MailHog для локального тестирования электронной почты (пошаговое руководство)

Салман Равуф | Обновлено: 15 июля 2024 года



Тестирование электронной почты в локальной среде разработки может быть непростой задачей. Очень сложно понять, доходят ли исходящие письма с вашего сайта или веб-приложения до почтового ящика получателя. На помощь приходит MailHog!

MailHog – это инструмент для тестирования электронной почты, который позволяет легко установить и настроить локальный почтовый сервер. MailHog создает поддельный SMTP-сервер. Вы можете настроить свои любимые веб-приложения на использование SMTP-сервера MailHog для отправки и получения электронных писем.

Например, вы можете настроить локальный сайт на WordPress для использования MailHog для доставки электронной почты. Именно так работает [DevKinsta](#) со своим встроенным SMTP-сервером для локального тестирования электронной почты.

В этой статье вы узнаете все тонкости работы с MailHog, в том числе об установке и настройке, о том, как тестировать электронные письма локально, и многом другом!

Готовы приступить? Поехали!

Оглавление

[Что такое MailHog?](#)

[Зачем нужен MailHog](#)

[Возможности MailHog](#)

[Развертывание MailHog на различных платформах](#)

[Настройка MailHog с помощью mhsendmail](#)

[Локальное тестирование электронной почты с помощью MailHog](#)

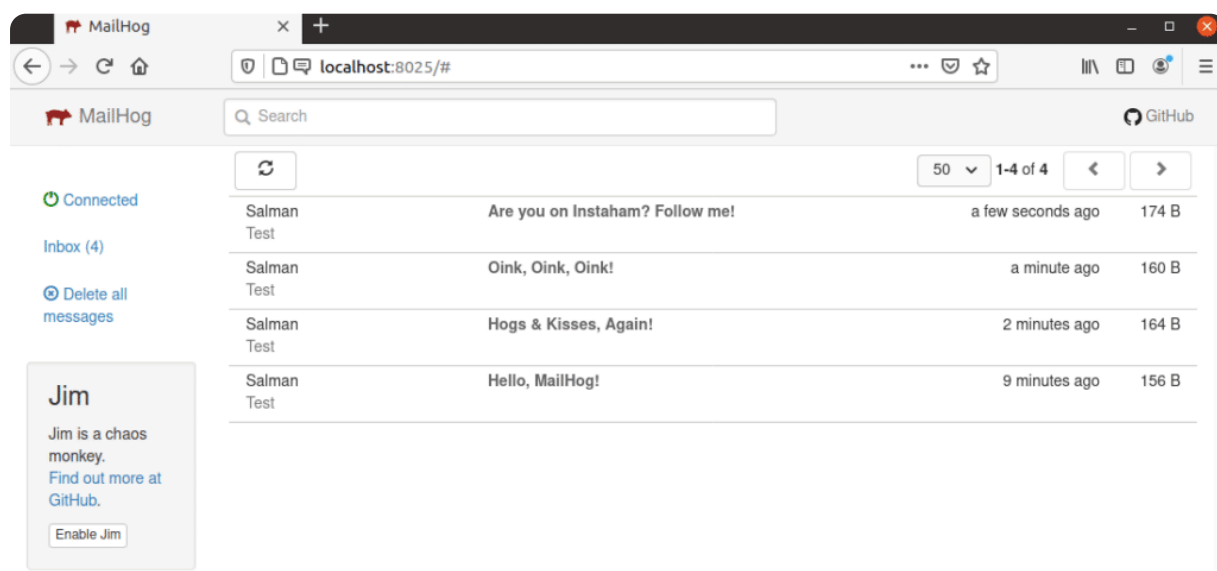
[DevKinsta и MailHog: простое локальное тестирование электронной почты](#)

Скачать статью в формате PDF

Предпочитаете посмотреть [видеоверсию?](#)



Что такое MailHog?



— Веб-интерфейс MailHog

MailHog – это инструмент для тестирования электронной почты с открытым исходным кодом, предназначенный в первую очередь для разработчиков. Он позволяет более эффективно тестировать возможности отправки и получения электронной почты в веб-приложении.

MailHog написан на языке программирования Go и может работать в различных операционных системах, включая Windows, Linux, FreeBSD и

macOS. MailHog разработан Иэном Кентом и распространяется по лицензии MIT, поэтому вы можете [использовать его бесплатно](#) в личных и коммерческих целях.

Но зачем вам вообще нужен MailHog?

G2's top pick for WordPress hosting 

Migrate your site and unlock the power of our premium-tier network.

Sign up for free

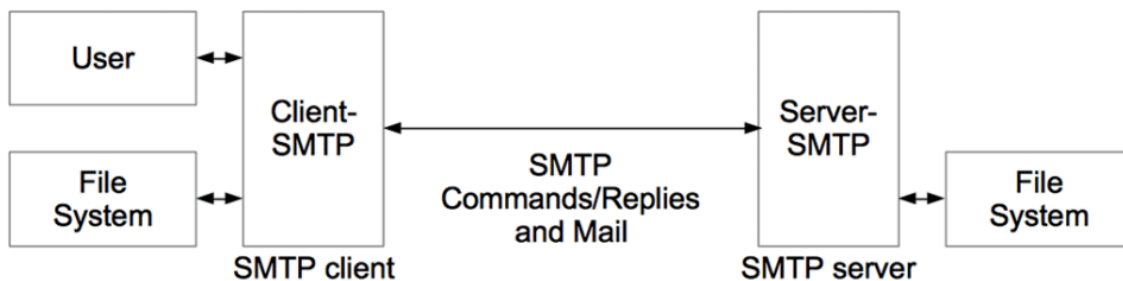
Зачем нужен MailHog

MailHog решает многие основные проблемы, связанные с тестированием электронной почты.

Допустим, вы разрабатываете сайт на WordPress в локальной среде разработки. Если вы хотите протестировать контактную форму или любой другой исходящий почтовый сервис, [это может оказаться непростой задачей](#).

Обычно эту задачу выполняет SMTP-сервер веб-приложения по умолчанию. В локальной среде разработки он почти всегда не работает по нескольким причинам.

Вы можете прочитать наше подробное руководство по [бесплатным SMTP-серверам](#), чтобы узнать больше о том, как работает SMTP-сервер.



– Как работает SMTP-сервер (Источник: Wikimedia Commons)

Во-первых, вам нужно настроить операционную систему, [веб-сервер](#) и [веб-приложение](#), чтобы отправлять электронные письма в локальной среде. Затем вам нужно будет убедиться, что письма успешно доходят до [почтового ящика](#) получателя, иначе вы потратите время впустую (и место в почтовом ящике).

Кроме того, существует проблема использования реального адреса электронной почты для тестирования. Это может подорвать доверие к вашему [частному адресу электронной почты](#).

MailHog решает все вышеперечисленные проблемы. Он создает поддельный SMTP-сервер, с помощью которого вы можете настроить свое веб-приложение для отправки и получения электронных писем. Он даже хранит отправленные и полученные письма в удобном веб-интерфейсе [UI](#), так что вы можете просматривать их, как в обычном почтовом ящике.

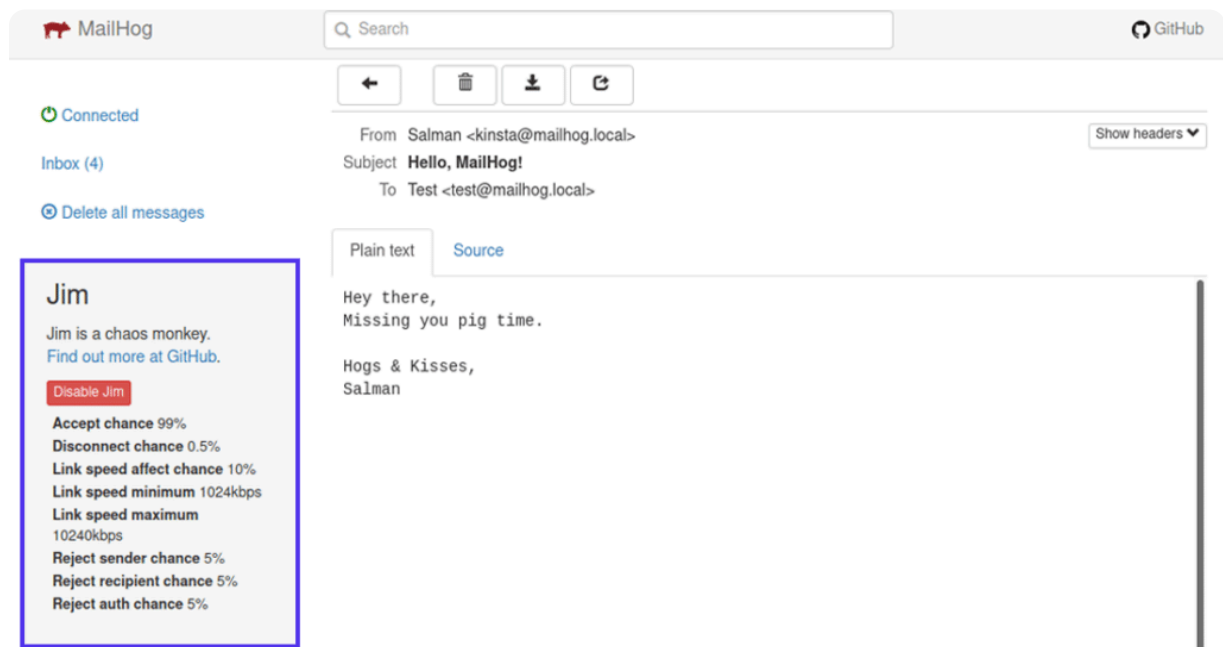
Наконец, MailHog позволяет получать эти электронные письма с помощью [API](#) для [автоматизированного тестирования](#). Это мощная функция, и вы сами решаете, как ее использовать.

Возможности MailHog

MailHog обладает множеством встроенных функций. Вот некоторые из них:

- **Легкая и портативная версия:** для использования MailHog не нужно его устанавливать. Это одно из главных преимуществ MailHog. Благодаря этому его можно запускать практически в любой операционной системе или на любом веб-сервере.

- **Расширенная поддержка SMTP:** MailHog не только позволяет настроить поддельный SMTP-сервер, но и настроить ESMTP-сервер (расширенный SMTP). Кроме того, он поддерживает SMTP AUTH и PIPELINING.
- **Поддержка API:** Вы можете использовать встроенный [HTTP API](#) (или JSON API) MailHog для получения, просмотра и удаления электронных писем.
- **Тестирование на отказ в работе:** Вы можете пригласить [Джима](#), «Хаос-обезьянку» MailHog (да, так его и зовут), чтобы протестировать доставку писем в вашем веб-приложении. Он случайным образом создает распространенные проблемы с доставкой писем, такие как отказ в подключении/аутентификации и ограничение скорости подключения.
- **Обновления в режиме реального времени:** MailHog использует веб-интерфейс [EventSource](#) для мгновенного обновления данных.
- **Сбор и хранение электронных писем:** Собранные электронные письма могут храниться в оперативной памяти и в [базе данных](#) MongoDB, а также в файловой системе.
- **Загрузка вложений:** MailHog поддерживает загрузку отдельных частей MIME.
- **Веб-интерфейс:** с помощью веб-интерфейса MailHog можно просматривать электронные письма в виде обычного текста, [HTML](#) или исходного кода. Он также поддерживает составные MIME-сообщения и закодированные заголовки.
- **Аутентификация:** MailHog поддерживает [базовую HTTP-аутентификацию](#) как для веб-интерфейса, так и для API.
- **Отправка перехваченных писем:** вы можете настроить MailHog на отправку перехваченных писем на реальные SMTP-серверы для доставки.



– Jim is MailHog's "Chaos Monkey"

Apart from the above features, MailHog currently has two client libraries for [PHP](#) and [NodeJS](#) environments to interact with the MailHog API easily.

Deploying MailHog on Various Platforms

As mentioned before, MailHog doesn't need to be installed. However, you still have to download it and set it up in your development environment.

Let's get started with downloading MailHog. Then we'll show you how to set it up on various platforms. There are many ways you can go about doing it.

MailHog on Windows

First, head to [MailHog releases](#) page on GitHub to download its latest stable version for Windows. After downloading MailHog, you can run it as is without any external dependencies.

Latest release

v1.0.1
0441dd4

Compare

Mailhog 1.0.1

tyndyll released this on Aug 11, 2020

This is a minor bump, to get the project moving again, and to release some small changes including

- Change to official Golang docker image
- Documentation updates
- Addition of Ubuntu snap

The target cadence going forward will be a minor release every three months, with features and enhancements. Security or bug fixes will be released as path releases.

Assets 8

MailHog_darwin_amd64	12.7 MB
MailHog_linux_386	11.4 MB
MailHog_linux_amd64	12.7 MB
MailHog_linux_arm	11.3 MB
MailHog_windows_386.exe	11.2 MB
MailHog_windows_amd64.exe	12.5 MB
Source code (zip)	

– MailHog GitHub releases page

Note: As of writing this article, [MailHog 1.0.1](#) is the latest release version. You can download it precompiled for Windows, Linux, and Darwin (macOS) platforms. If you're using a niche platform, you can also download its [source code](#) and compile it for your specific use case.

As MailHog for Windows is a simple self-executable program, you can just run the downloaded `.exe` file. If you're presented with any security alerts, make sure to allow MailHog to run on your [localhost environment](#) without [firewall](#) restrictions.

Here's how MailHog looks on Windows 10.

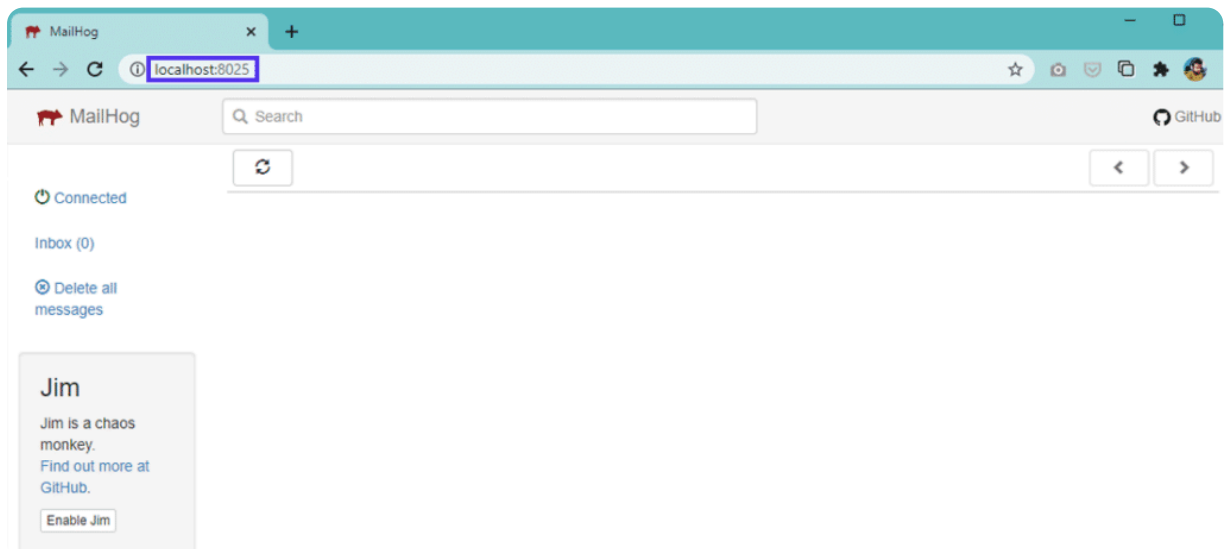
```
C:\Users\salmr\Desktop\Kandy\08_Mailhog\MailHog_windows_amd64.exe
2021/02/17 04:31:05 Using in-memory storage
2021/02/17 04:31:05 [SMTP] Binding to address: 0.0.0.0:1025
[HTTP] Binding to address: 0.0.0.0:8025
2021/02/17 04:31:05 Serving under http://0.0.0.0:8025/
Creating API v1 with WebPath:
Creating API v2 with WebPath:
[APIv1] KEEPALIVE /api/v1/events
```

– MailHog running on Windows 10

You can see that the SMTP server is on port `1025` and the HTTP server is on port `8025`.

Nothing too fancy yet, but it's a confirmation that MailHog is running on your system.

Now, you can visit `localhost:8025` or `127.0.0.1:8025` in your web browser to view the MailHog Web UI.



– MailHog Web UI on Windows

You cannot find any email listed here yet, as we haven't configured any of our web applications to use MailHog. We'll cover that later in this article.

MailHog on Linux

If you're using Linux (Ubuntu/Debian), here's how you can deploy MailHog:

```
sudo apt-get -y install golang-go
sudo apt-get install git
go install github.com/mailhog/MailHog
```

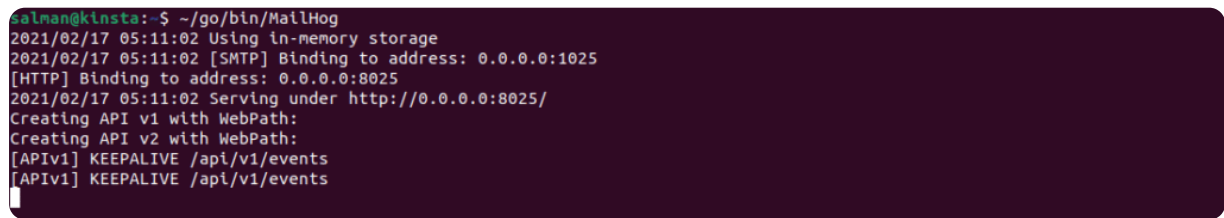
The first Linux command installs the Golang programming language on your development environment (if you don't have it already). You can skip this line if you've installed Go already. The next command installs `git` on your machine (you can skip this too if it's already installed).

The third command uses Go's directive to get MailHog's latest stable release from its [GitHub](#) repository.

Next, you can run `/path/to/MailHog` through your [terminal](#) to start MailHog. The exact path may vary depending on your Linux flavor. As Go's bin files on Ubuntu are stored under the `~/go/bin/` directory, you can run the below code to start MailHog:

```
~/go/bin/MailHog
```

That's pretty much it. MailHog is now running on your Linux machine. This is how it looks.

A terminal window with a dark purple background. The text shows the command to run MailHog and its output logs, including timestamps, storage type, and binding addresses for SMTP and HTTP.

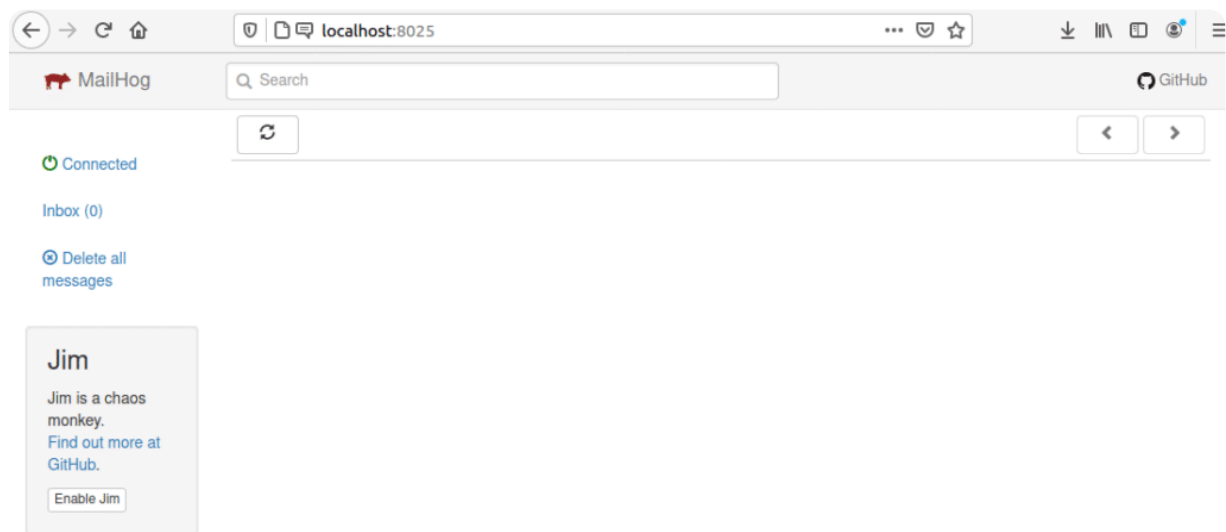
```
salman@kinsta:~$ ~/go/bin/MailHog
2021/02/17 05:11:02 Using in-memory storage
2021/02/17 05:11:02 [SMTP] Binding to address: 0.0.0.0:1025
[HTTP] Binding to address: 0.0.0.0:8025
2021/02/17 05:11:02 Serving under http://0.0.0.0:8025/
Creating API v1 with WebPath:
Creating API v2 with WebPath:
[APIv1] KEEPALIVE /api/v1/events
[APIv1] KEEPALIVE /api/v1/events
```

– MailHog running on Linux (Ubuntu)

If you want to start MailHog automatically on every login, you can set it to start as a daemon service using supervisord, upstart, etc. You can also use the [init script](#) or [Ansible role](#) provided by [geerlingguy](#) to do the same.

Just like with Windows, you can see that the SMTP server starts on port `1025` and the HTTP server starts on port `8025` .

Visit `localhost:8025` or `127.0.0.1:8025` in your web browser to see the MailHog Web UI.



– MailHog Web UI on Linux (Ubuntu)

Save big with Kinsta

Get \$300 in integrations, including CDN and DDoS protection.

Start for free

MailHog on macOS

Apple users can install MailHog using Homebrew. Open up your terminal and run the following command:

```
brew install mailhog
```

That'll install MailHog on your macOS. Next, you can start MailHog with:

```
mailhog
```

If you want MailHog to launch automatically on login and run as a background service, you can run the following command instead:

```
brew services start mailhog
```

That's pretty much it. MailHog is now installed on your macOS. You can view its Web UI by visiting `127.0.0.1:8025` or `localhost:8025` in your browser.

MailHog on Docker

Docker is one of the easiest ways to set up MailHog across any platform.

First, make sure that you have Docker installed on your machine. Next, you can start MailHog directly from Docker Hub by running the following command:

Try Kinsta Now - First Month Free!

```
docker run -d -p 1025:1025 -p 8025:8025 mailhog/mailhog
```

Let's break down the above command and its directives:

- `docker run` starts a docker container.
- `-d` directive runs the docker container in the background.
- `-p 1025:1025` and `-p 8025:8025` directives expose MailHog's default SMTP ports to your local ones.
- `mailhog/mailhog` grabs the latest MailHog build from Docker Hub.

Alternately, if you want to mount MailHog's `Maildir` to your local filesystem (instead of inside a Docker container), you can run the following command:

```
docker run -d -e "MH_STORAGE=maildir" -v $PWD/maildir:/maildir -p
```

Like before, you can visit `localhost:8025` or `127.0.0.1:8025` in your browser to use MailHog Web UI.

MailHog on Docker Compose

Docker Compose is an extension of Docker that makes it easy to run multiple Docker containers with a single command. You do this by defining all the containers you want to run in a YAML file.

By default, Docker Compose requires you to name this file as

```
docker-compose.yaml .
```

To run MailHog with Docker Compose, here's how your `docker-compose.yaml` file should look:

```
version: "3"
services:
  mailhog:
    image: mailhog/mailhog:latest
    restart: always
    ports:
      - 1025:1025
      - 8025:8025
```

Inside the folder where this file resides, run the following command to start MailHog in the background:

```
docker-compose up -d mailhog
```

It works exactly like with Docker. Except here, you don't have to mention many directives as you've defined them already.

You can now enter `localhost:8025` or `127.0.0.1:8025` in your browser to view MailHog Web UI.

In addition to the above methods, MailHog's GitHub repo also includes a guide for deploying MailHog on [AWS Elastic Beanstalk](#).

Configuring MailHog with mhsendmail

MailHog [redirects](#) emails from web apps to its SMTP server using [mhsendmail](#), a `sendmail` replacement created specifically for MailHog. It ensures that your web app's emails are sent and received using `localhost:1025` SMTP server, which MailHog is configured to handle by default.

To get started, you need to download and install `mhsendmail`. Depending on your platform, you can approach this in various ways.

For instance, if you're using Ubuntu, you can download `mhsendmail` from its [GitHub repository](#) by using the following command:

```
wget https://github.com/mailhog/mhsendmail/releases/download/v0.2.
```

Next, make this file executable and move it to your user directory. To do this, use the following command:

```
sudo chmod +x mhsendmail_linux_amd64
sudo mv mhsendmail_linux_amd64 /usr/local/bin/mhsendmail
```

Alternately, you can also get `mhsendmail` on Golang or Homebrew by running this command:

```
go install github.com/mailhog/mhsendmail
```

Make sure that Go is installed before you run the above command. If you're on Linux, refer to the *MailHog on Linux* section above. For installing Go with Homebrew, run the following command:

```
brew install go
```

With `mhsendmail` downloaded, you can now proceed with installing it:

```
go install github.com/mailhog/mhsendmail
```

Next, you need to configure your web app to use MailHog's `mhsendmail` as its email routing service. Most Linux systems have it set to use `sendmail` by default.

You can do this by editing your `php.ini` file. On most servers, you can [find the php.ini file](#) in the root directory. You can also execute the following command to find where it is:

```
php -i | grep 'php.ini'
```

For Linux machines running Ubuntu, you need to set the value of

`sendmail_path` to:

```
sendmail_path = /usr/local/bin/mhsendmail
```

Instead of the above value, you can also use the value shown below to get the same result:

```
sendmail_path = /usr/sbin/sendmail -S mail:1025
```

If you used Go to install `mhsendmail`, or if you're using any other variant of Linux, the path may vary slightly depending on your platform/configuration. Make sure to link to the proper `mhsendmail` directory.

Mac users need to edit their `Postfix` config file to set up MailHog properly. You can use the following Postfix configuration:

```
myhostname = localhost  
relayhost = [127.0.0.1]:1025
```

For WordPress sites, you can use the free [MailHog for WordPress](#) plugin to route your site's emails to MailHog. To use it, install and activate the plugin in your WordPress admin dashboard. A better alternative is to use [DevKinsta](#). We'll cover it in depth later.

Read the [MailHog documentation](#) for information about how to set it up on various other platforms/configurations.

Additional MailHog Configuration Options

You can configure MailHog using its various [environment variables](#) or [command line](#) options.

For example, you can change its default hostname from `mailhog.example` to a custom name by using the `MH_HOSTNAME` [environment](#) variable or the `-hostname` command line option.

For more information, you can check out the [MailHog configuration options](#) documentation.

Local Email Testing with MailHog

Finally, let's [test](#) our MailHog setup to see whether it's configured properly.

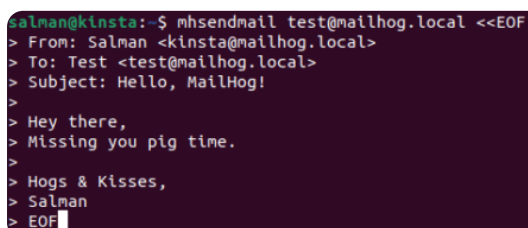
You can send an email with Mailhog's CLI by using the following `mhsendmail` directives:

```
mhsendmail test@mailhog.local <<EOF
From: Salman <kinsta@mailhog.local>
To: Test <test@mailhog.local>
Subject: Hello, MailHog!
```

```
Hey there,
Missing you pig time.
```

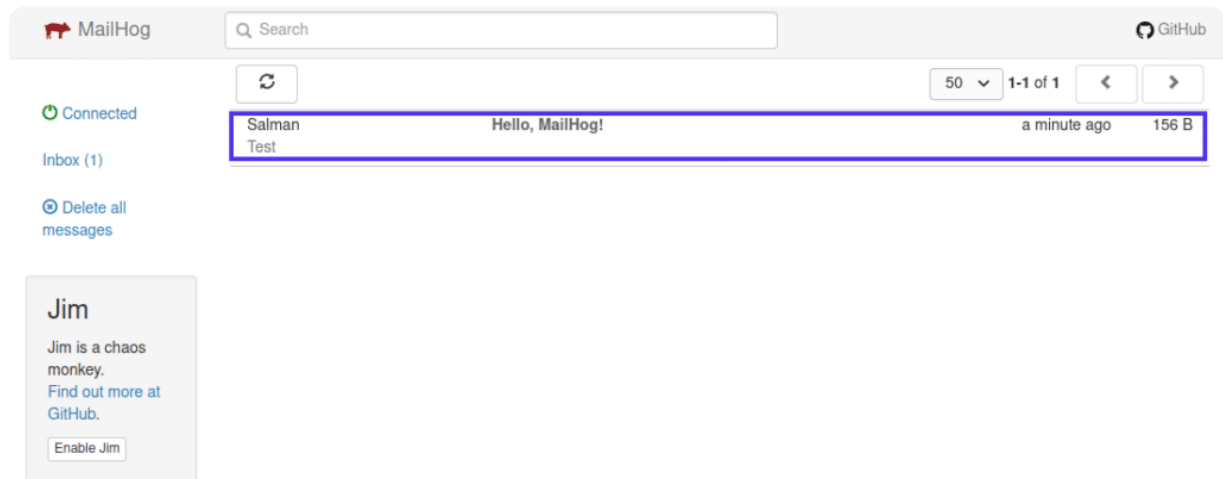
```
Hogs & Kisses,
Salman
EOF
```

In the above code, the `<<EOF` directive sets a custom delimiter so that you can enter the test email details interactively inside the terminal. It executes all the input when you type in the delimiter again. Here, it sends a mail using the `mhsendmail` command.



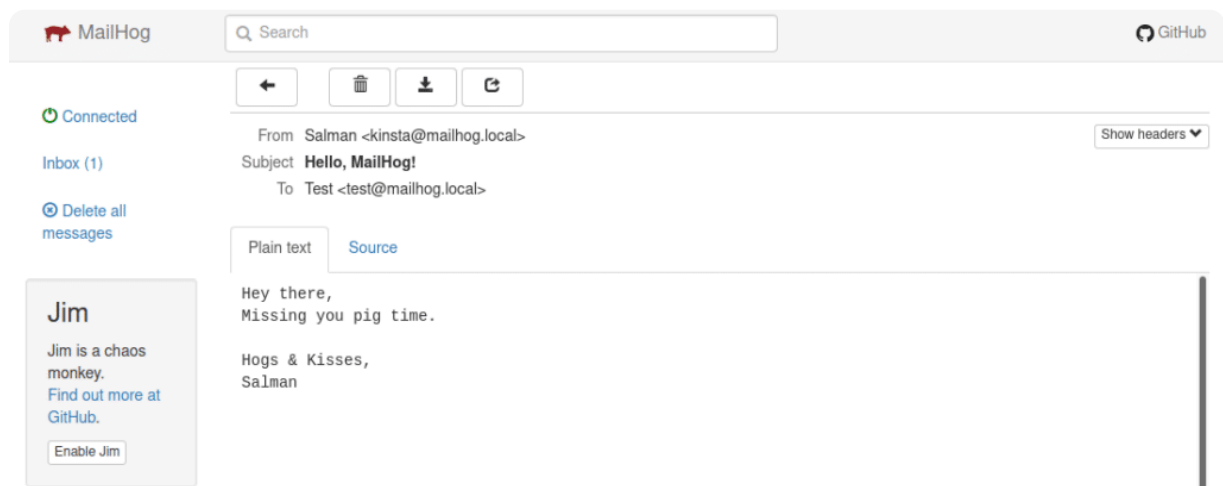
```
salman@kinsta:~$ mhsendmail test@mailhog.local <<EOF
> From: Salman <kinsta@mailhog.local>
> To: Test <test@mailhog.local>
> Subject: Hello, MailHog!
>
> Hey there,
> Missing you pig time.
>
> Hogs & Kisses,
> Salman
> EOF
```

You can verify whether the email was delivered successfully by visiting the MailHog Web UI.



– The test email has been delivered successfully

You can also click on the test email to verify its body content.



– Verifying the test email's body content

That's a wrap! We can now conclude that MailHog has been configured successfully.

Adding to the above method, there are other ways you can send a test email. For instance, on macOS, you can send a test email by running the following command in your terminal:

```
date | mail -s "Hello, MailHog!" test@mailhog.local
```

On Windows, you can send an email with PowerShell by executing the following code:

```
Send-MailMessage -To "test@mailhog.local" -From "salman@kinsta.com"
```

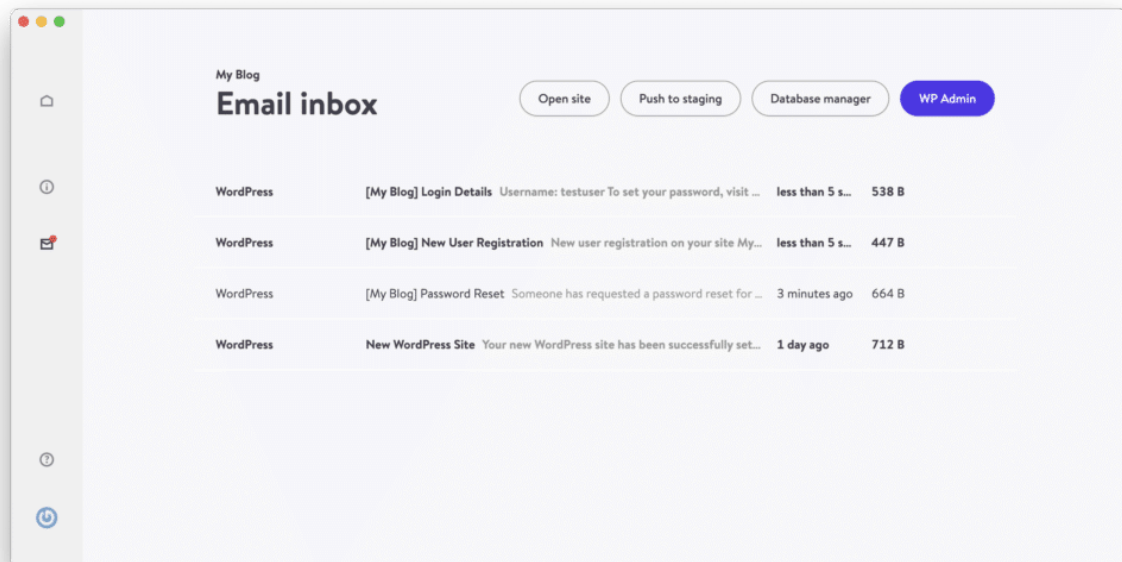
You can also define a [PHP](#) script with all the mail data and use PHP's `mail()` function to send a test email. To do that, define your PHP script like below and execute it from the terminal:

```
<?php
$to = "test@mailhog.local";
$subject = "Hey, I'm Pi Hog Pi!";
$body = "Hello, MailHog!";
$headers = "From: pihogpi@kinsta.com" . "\r\n";
mail($to,$subject,$body,$headers);
?>
```

Lastly, you can use MailHog API to test whether an email has been delivered successfully. If you're working exclusively in a non-GUI environment (e.g. Linux terminal, Windows PowerShell), the MailHog API super handy to list, [retrieve and delete emails](#).

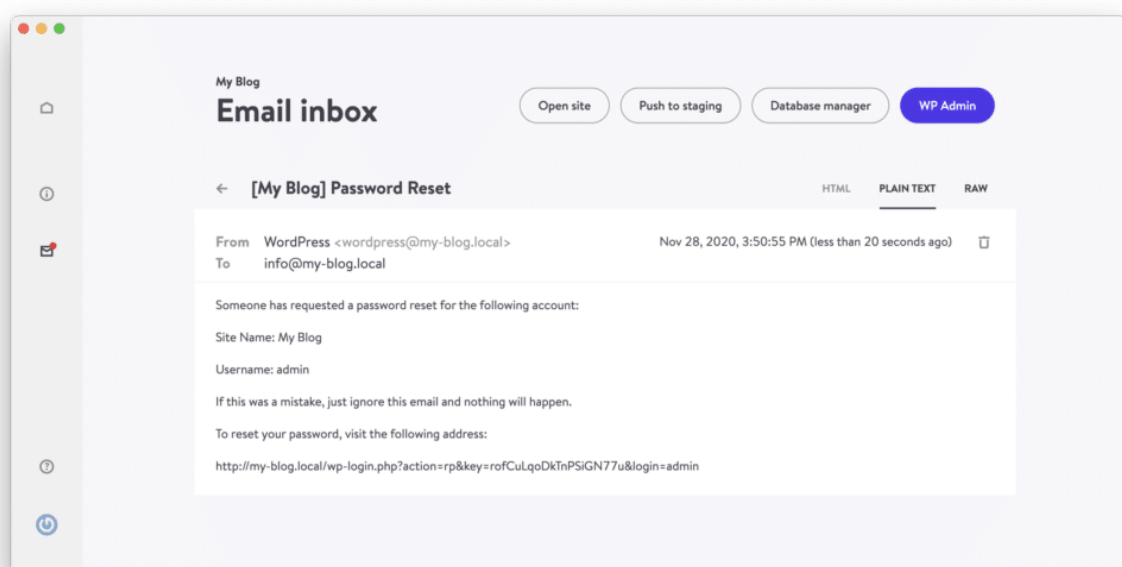
DevKinsta and MailHog: Hassle-Free Local Email Testing

Our local WordPress development tool, [DevKinsta](#), uses MailHog to power its email testing features. As it also Docker to containerize individual sites, you'll have a separate MailHog inbox for every local site you're developing.



– DevKinsta's 'Email Inbox' is powered by MailHog

Like MailHog Web UI, you can use DevKinsta's local email inbox to view and manipulate emails (delete, mark as read, view source).



– View and manipulate emails in DevKinsta

You can also check for DevKinsta's MailHog [logs](#) by running the following Docker command:

```
docker logs -f devkinsta_mailhog
```

Head to [DevKinsta's knowledgebase](#) section for more information.

Summary

MailHog makes email testing a breeze. It's a great, practical solution to test your website's or web app's emails. Moreover, MailHog is simple to set up on almost all popular platforms and has an easy-to-use Web UI.

Like MailHog, DevKinsta is a great way to get started with local WordPress development. Not only does it include MailHog for testing emails locally, but it also comes preloaded with [Adminer](#) for easy database management.

With MailHog (and DevKinsta) on your side, you'll find that testing emails locally has never been easier.

If you want to share your experience or challenges with testing emails locally, please drop us a line in the comment section below.

Power your site with Kinsta's Managed WordPress hosting, crafted for speed, security, and simplicity. With Kinsta, you get:

- Effortless control in the MyKinsta dashboard
- Unlimited free migrations, handled by our expert migrations team
- 24/7/365 support from WordPress experts
- Powerful cloud infrastructure and cutting-edge servers
- Enterprise-grade security through Cloudflare integration
- Global reach with 27 data centers

Enjoy your [first month free!](#)



Salman Ravoof

Salman Ravoof is a self-taught web developer, writer, creator, and a huge admirer of Free and Open Source Software (FOSS). Besides tech, he's excited by science, philosophy, photography, arts, cats, and food. Learn more about him on his [website](#), and connect with Salman on [Twitter](#).



Related Articles and Topics

Kinsta

Managed Hosting for WordPress

Site information

Basic details

Location	Site name	Domain
London, UK	WordPress	WordPress.com

Environment details

Plan	Site ID	IP address	IP address for external connectivity	Tier
WordPress.com, 200GB	101101101-01	101.101.101.101	101.101.101.101	1

Visits

Aug 20 - Aug 21

24,345

Overall transaction time

The best home for modern WordPress sites

High-performance hosting for WordPress

200% faster WordPress sites. Global data centres. Free SSL & DDoS protection. 24/7/365 expert support. Switch to Kinsta and get your first month free!

4 min read · April 24, 2026 · Page · Video



How to Install WordPress Locally (Windows, macOS, Linux)

Sometimes it can be convenient to work on your own machine. Check out how to install WordPress locally using DevKinsta, XAMPP, WAMP, or MAMP.

30 min read · August 2, 2024 · Blog · DevKinsta · Localhost · WordPress Development

Local Development

WordPress

Database

Browsers

PHP

Get expert support whenever you need it

Under 2-minute response time. 98% satisfaction rate. Included in all WordPress plans.

Migrate for free

Products

Feature highlights

Use cases

Resources

Company

We take security and privacy seriously.

[Trust Center](#) · [Privacy Policy](#) · [Your Privacy Choices](#)

SOC 2	ISO		
Type II	27001	GDPR	CCPA



English

© 2013 - 2026 Kinsta Inc. All rights reserved. Kinsta®, MyKinsta®, DevKinsta®, and Sevala® are trademarks owned by Kinsta Inc.

The WordPress® trademark is the intellectual property of the WordPress Foundation, and the Woo® and WooCommerce® trademarks are the intellectual property of WooCommerce, Inc. Uses of the WordPress®, Woo®, and WooCommerce® names in this website are for identification purposes only and do not imply an endorsement by WordPress Foundation or WooCommerce, Inc. Kinsta is not endorsed or owned by, or affiliated with, the WordPress Foundation or WooCommerce, Inc.